

CS 300 Project One: Pseudocode and Runtime Analysis

Course Object

Course

courseNumber

courseTitle

prerequisites

The course object stores one line from the input file. The first field is the course number, the second field is the course title, and any remaining fields are prerequisites.

Vector Pseudocode

```
void loadCourses(Vector<Course> courses, String fileName) {
```

```
    open file
```

```
    if file does not open
```

```
        print error message
```

```
        return
```

```
    while file has another line
```

```
        read line
```

```
        split line by comma
```

```
        if line has fewer than 2 fields
```

```
            print formatting error
```

continue

create Course object

course.courseNumber = first field

course.courseTitle = second field

for each remaining field

add field to course.prerequisites

add course to vector

close file

}

void printCourseList(Vector<Course> courses) {

sort courses by courseNumber from lowest to highest

for all courses

print course.courseNumber and course.courseTitle

}

void searchCourse(Vector<Course> courses, String courseNumber) {

for all courses

if the course is the same as courseNumber

```
    print out the course information

    for each prerequisite of the course
        print the prerequisite course information

    return

print course not found
}
```

Hash Table Pseudocode

```
void loadCourses(HashTable<Course> courses, String fileName) {
    open file

    if file does not open
        print error message
        return

    while file has another line
        read line
        split line by comma

        if line has fewer than 2 fields
```

print formatting error

continue

create Course object

course.courseNumber = first field

course.courseTitle = second field

for each remaining field

add field to course.prerequisites

insert course into hash table using courseNumber as key

close file

}

void printCourseList(HashTable<Course> courses) {

create temporary vector

for each bucket in hash table

for each course in bucket

add course to temporary vector

sort temporary vector by courseNumber from lowest to highest

```

    for all courses in temporary vector
        print course.courseNumber and course.courseTitle
    }

void searchCourse(HashTable<Course> courses, String courseNumber) {
    calculate hash key using courseNumber

    search the bucket at the hash key

    if the course is found
        print out the course information

        for each prerequisite of the course
            print the prerequisite course information

        return

    print course not found
}

```

Binary Search Tree Pseudocode

```

void loadCourses(Tree<Course> courses, String fileName) {
    open file

```

if file does not open

 print error message

 return

while file has another line

 read line

 split line by comma

 if line has fewer than 2 fields

 print formatting error

 continue

 create Course object

 course.courseNumber = first field

 course.courseTitle = second field

 for each remaining field

 add field to course.prerequisites

 insert course into binary search tree using courseNumber

```
    close file
}

void printCourseList(Tree<Course> courses) {
    perform in-order traversal

    visit left node

    print course.courseNumber and course.courseTitle

    visit right node
}

void searchCourse(Tree<Course> courses, String courseNumber) {
    start at the root node

    while current node is not null
        if the course is the same as courseNumber
            print out the course information

            for each prerequisite of the course
                print the prerequisite course information

            return

        else if courseNumber is less than current node courseNumber
            move to the left child
```

```
    else  
        move to the right child  
  
    print course not found  
}
```

Menu Pseudocode

display menu

1. Load Data Structure
2. Print Course List
3. Print Course
9. Exit

while user choice is not 9

get user choice

if choice is 1

ask user for file name

load file data into the data structure

if choice is 2

if data is loaded

 print alphanumeric course list

else

 print message to load data first

if choice is 3

 if data is loaded

 ask user for course number

 search course and print title and prerequisites

 else

 print message to load data first

if choice is 9

 exit program

Runtime Analysis

Vector Load Runtime

Code	Line	Cost	# Times Executes	Total Cost
open file	1		1	1
while file has another line	1		n	n
read line	1		n	n
split line by comma	1		n	n
check formatting error	1		n	n
create Course object	1		n	n
add prerequisites	1		n	n
add course to vector	1		n	n
close file	1		1	1

Total Cost: $7n + 2$

Runtime: $O(n)$

Hash Table Load Runtime

Code	Line	Cost	# Times Executes	Total Cost
open file	1		1	1
while file has another line	1		n	n
read line	1		n	n
split line by comma	1		n	n
check formatting error	1		n	n
create Course object	1		n	n
add prerequisites	1		n	n
insert into hash table	1		n	n
close file	1		1	1

Total Cost: $7n + 2$

Runtime: $O(n)$

Binary Search Tree Load Runtime

Code	Line	Cost	# Times Executes	Total Cost
open file	1		1	1
while file has another line	1		n	n
read line	1		n	n
split line by comma	1		n	n
check formatting error	1		n	n
create Course object	1		n	n
add prerequisites	1		n	n
insert into tree	$\log n$		n	$n \log n$
close file	1		1	1

Total Cost: $n \log n + 6n + 2$

Runtime: $O(n \log n)$

Evaluation

The vector data structure is simple and easy to understand. It stores all courses in one list and uses $O(n)$ memory. Loading the file and creating course objects takes $O(n)$ time. The disadvantage of the vector is that searching for one course takes $O(n)$ time because the program

may need to check every course. Also, to print the course list in alphanumeric order, the vector must be sorted first, which takes $O(n \log n)$.

The hash table is efficient for searching. It also uses $O(n)$ memory, and loading the file takes $O(n)$ time on average. A course can usually be found in $O(1)$ time if the hash function distributes the courses well. The disadvantage is that a hash table does not store the courses in alphanumeric order. To print the full course list in order, the program must copy the courses into a temporary vector and sort them, which takes $O(n \log n)$.

The binary search tree uses $O(n)$ memory. Loading the file takes $O(n \log n)$ time on average because each course must be inserted into the tree. The advantage of the binary search tree is that it keeps courses organized by course number. It can print the full course list in alphanumeric order by using in-order traversal. Searching for one course is $O(\log n)$ on average. The disadvantage is that if the tree becomes unbalanced, the worst-case search and insert time can become $O(n)$.

Recommendation

I recommend using the binary search tree for the ABCU advising program. The main requirements are to print all Computer Science courses in alphanumeric order and to print the title and prerequisites for a selected course. The binary search tree is the best fit because it naturally supports sorted output through in-order traversal. It also provides efficient average search performance for looking up an individual course. Although the tree can become unbalanced in the worst case, it still best matches the advisor's requirement to print courses in alphanumeric order.